

Universitatea din București
Facultatea de Matematică și Informatică
Specializarea Informatică

Raport Competiție Kaggle - Detectția obiectelor în Semnale Radio

Curs: Inteligență Artificială, Învățare Automată

Student: Drăgunoi Miruna,
Grupa 244

Cuprins

Contextul problemei	3
Analiza datelor și observații	4
Preprocesarea imaginilor	5
Augmentarea datelor și alegeri	6
Reprezentarea trăsăturilor	7
Arhitectura CNN - Rețeaua convoluțională LineCNN	9
Modele de clasificare și strategia de ensemble folosită	11
Detalii despre antrenarea și strategia OOF	12
Variantele de rezolvări implementate și rezultatele obținute	14
Abordări ale problemei nereușite	19
Discuție finală	20
Anexă	21
Bibliografie	22

Contextul problemei

Acest raport reprezintă abordarea mea pentru competiția Kaggle intitulată - “Signal Object Detection”. Obiectivul problemei era de a detecta/număra obiectele prezente în imagini de semnale radio care erau afectate de zgomot. Aveam un set de date de 15.500 imagini etichetate pentru antrenare și un număr de 5.500 imagini de test. Rezoluția nativă a unei imagini era de 55 x 128 pixeli în format RGBA. Un aspect important din problemă este logica etichetelor, fiecare aparținând uneia din cele 5 clase, acestea reprezentând numărul de obiecte detectate în imagini.

Pentru a începe să rezolv problema, primul aspect pe care a trebuit să-l tratez a fost să înțeleg dacă problema este una de clasificare sau de regresie. Inițial, m-am ghidat după clase, ce sunt reprezentate de numere întregi consecutive de la 1 la 5, așa că regresia a părut o opțiune convenabilă, deoarece erorile între clasele apropiate ar fi fost mai puțin grave decât cele între clasele îndepărtate. Pe parcurs, când am început să implementez codul pentru problemă, am testat empiric ambele variante, însă cu modelul de regresie, fiind un CNN antrenat cu MSE pe etichetele continue și rotunjit la cel mai apropiat întreg, am obținut o acuratețe de 0.5535 la rularea locală și 0.61236 pe Kaggle, iar pentru modelul în care am folosit clasificare am obținut o acuratețe de 0.7172. Așadar, am realizat că atât timp cât metrica de evaluare este acuratețea, rezolvarea problemei prin clasificare este o abordare mai reușită, deoarece regresia nu este penalizată pentru anumite greșeli mai mici în spațiul continuu, dar acuratețea o să pedepsească în mod egal orice predicție care se dovedește a fi incorectă.

Încă un aspect pe care l-am observat în problemă a fost dezechilibrul distribuției claselor în setul de antrenare: clasa 1 are 3500 de exemple, iar clasele 2-5 au câte 3000 de exemple fiecare. Am rezolvat acest dezechilibru prin utilizarea ponderilor de clasă în funcția de pierdere (*class_weight='balanced'* în scikit-learn, respectiv *weight=class_weight* în *nn.CrossEntropyLoss*).

Pentru a avea o acuratețe cât mai bună pentru datele neobservate, adică o estimare cât mai onestă a performanței, întreaga strategie de validare pe care am construit-o s-a bazat în jurul tehnicii out-of-fold, și anume: am împărțit setul de antrenare stratificat în 5 folduri, iar pentru fiecare imagine din train predicția pe care am folosit-o la evaluare a provenit de la modelul care nu a văzut acea imagine la antrenare. Strategia pe care am folosit-o previne data leakage în momentul în care modele de stack, cum ar fi SVM sau MLP, sunt antrenate pe trăsăturile extrase din CNN.

Ulterior, după ce am testat 10 variante diferite ale pipeline-ului, soluția mea finală a obținut o acuratețe de 0.78545 pe leaderboard-ul public Kaggle, adică pe 25% din date, și 0.77478 pe leaderboard-ul privat, pe 75% din date.

Analiza datelor și observații

Înainte de a începe efectiv implementarea în cod, a trebuit să înțeleg mai mult logica imaginilor ca să pot alege corect strategia de modelare. Imaginile sunt de fapt spectrograme afișate cu un colormap violet-verde, în care obiectele ce trebuie numărate apar ca niște linii subțiri (aparent aproximativ 1-2 pixeli grosime) pe fundalul difuz. Din mai multe imagini analizate, am observat faptul că liniile de analizat au diferite orientări:

- linii verticale - adică semnale de frecvență constantă în timp
- linii diagonale cu pante variate - chirp-uri, adică semnale a căror frecvență variază în timp
- mai multe linii suprapuse sau intersectate - situații frecvente la clasele superioare de la 3 la 5.

Observațiile menționate anterior au fost un pas important în modelarea preprocesării, cât și în alegerea augmentărilor. Încă un aspect care m-a ghidat în preprocesare a fost determinată de faptul că, chiar dacă imaginile sunt salvate ca RGBA, ele provin de fapt dintr-un colormap aplicat unui semnal unidimensional, adică informația semantică este conținută în intensitatea spectrală, nu în componentele de culoare separate.

Această observație are două consecințe directe asupra modelării:

1. reducerea la grayscale nu pierde informație: toate cele trei canale (R, G, B) sunt funcții deterministe ale unei singure valori scalare. Așadar, conversia la luminanță reduce dimensionalitatea cu un factor de 3 fără pierdere de informație utilă.
2. manipularea culorilor (ColorJitter) ar dăuna foarte mult: orice modificare a contrastului sau a tonurilor RGB ar altera exact informația care indică prezența unui obiect.

Preprocesarea imaginilor

Aceasta este bazată pe trei pași secvențiali, fiind motivați de structura semnalului radio:

Pas 1 - Conversia de luminanță (grayscale)

- canalele R,G,B din imaginea originală nu codează informație independentă, ele fiind doar rezultatul unui colormap aplicat unui semnal unidimensional, și anume, intensitatea spectrală. Pentru a rezolva acest aspect, am recurs la reducerea la un singur canal care să păstreze toată informația utilă și reduce dimensionalitatea inputului pentru CNN.
- am aplicat transformarea $L = 0.299R + 0.587G + 0.114B$ (formula standard ITU-R 601), implementată automat prin `pil_img.convert('L')`. Rezultatul este de fapt o imagine pe un singur canal cu valori în intervalul $[0,255]$.

Pas 2 - Contrast stretching la percentilele 1% și 99%

- acesta este unul dintre aspectele cele mai importante din preprocesarea imaginilor. Liniile slabe, adică semnalele de joasă intensitate, se dovedesc a fi aproape invizibile pe imaginea brută din cauza fundalului. Am rezolvat această observație prin maparea intervalului [percentilă 1%, percentilă 99%] la intervalul $[0, 1]$. Astfel, valorile extreme, cum ar fi zgomotul rar și saturația, sunt eliminate, iar contrastul efectiv al liniilor crește. Această modificare a urcat singură acuratețea OOF de la aproximativ 0.72 la 0.7449 (v3 din experimentele documentate).

Pas 3 - Normalizarea per imagine (media 0 și deviația standard 1)

- am normalizat independent fiecare imagine astfel încât valorile pixelilor să aibă media 0 și deviația standard 1. Acest lucru o să uniformizeze scara între imagini și o să prevină ca CNN-ul să învețe tipare legate de intensitatea absolută a fundalului.

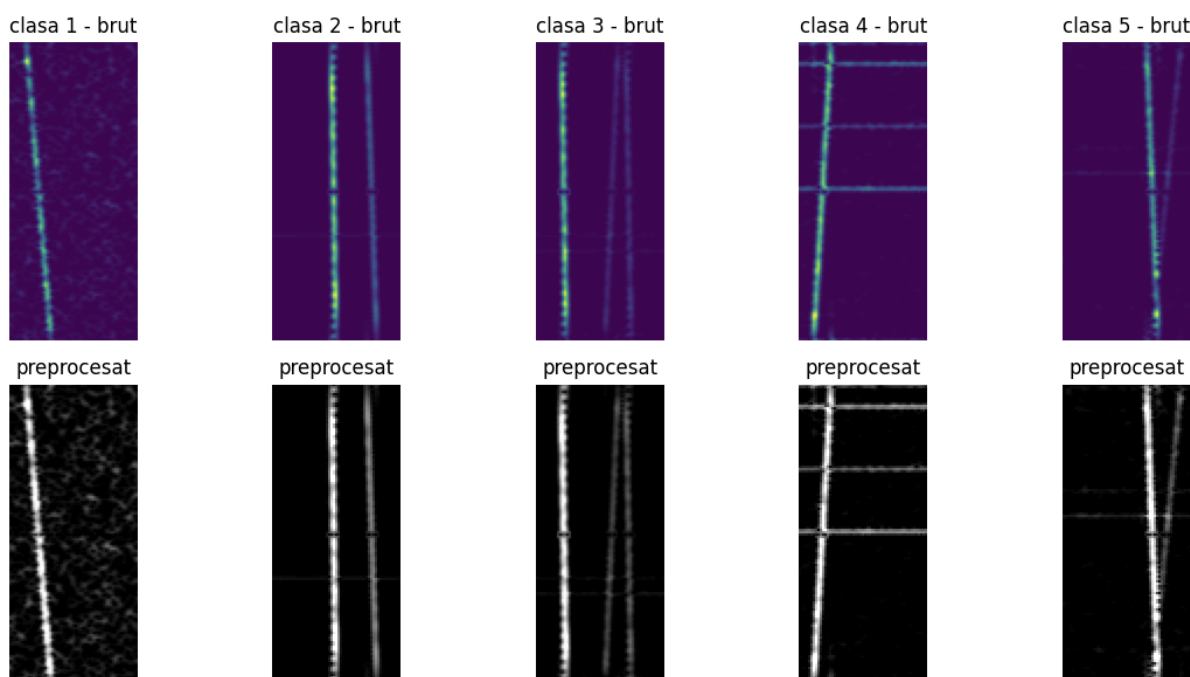


FIGURA 1: comparația vizuală înainte și după preprocesare cu câte un exemplu din fiecare din cele 5 clase

Augmentarea datelor și alegeri

Ținând cont că sarcina problemei se rezumă de fapt la una de numărare, înseamnă că orice transformare care ar putea schimba numărul efectiv de obiecte din imagine ar introduce zgomot în etichete (label noise). Din acest motiv am folosit două transformări:

- flip orizontal cu probabilitate de 0.5 - corespunde cu inversarea axei timpului în spectrogramă. Astfel, o linie diagonală cu pantă pozitivă devine o linie cu pantă negativă, dar o să rămână tot o linie, numărul de linii rămâne identic, deci numărul de obiecte nu s-ar schimba. Implementare în cod: `torch.flip(t, dims=[2])`.
- translație mică, aproximativ plus minus 3 pixeli pe ambele axe (realizată prin `torch.roll`) - realizează deplasarea liniilor în cadrul imaginii fără a exista riscul de a le elimina sau dubla. Am ales limita de plus minus 3 pixeli astfel încât liniile să rămână complet în interiorul imaginii.

Am avut grijă să nu folosesc următoarele augmentări, din cauza structurii datelor:

- flip vertical - axa verticală a spectrogramei reprezintă de fapt frecvența, care are sens fizic asimetric (frecvențele joase și înalte se comportă diferit). O frecvență joasă inversată ar deveni una înaltă, ceea ce poate să nu reprezinte o transformare validă a distribuției reale de test.
- RandomErasing - aceasta poate șterge accidental obiectul pe care eu trebuie să-l număr, transformând astfel o imagine etichetată cu 3 într-o imagine vizual asemănătoare cu eticheta 2. Asta mi-ar strica realitatea imaginilor, iar etichetele ar rămâne nemodificate.
- ColorJitter - într-o spectrogramă, culoarea codează de fapt magnitudinea semnalului. Dacă aș modifica contrastul sau luminozitatea aș altera exact informația care reprezintă un obiect.
- rotații mari - ar altera geometria liniilor în moduri care nu corespund cu distribuția reală de test.

Reprezentarea trăsăturilor

Soluția mea finală a problemei combină două surse complementare de informație, și anume, trăsăturile învățate automat de o rețea convoluțională și trăsăturile extrase manual pe structura semnalului. Prin concatenarea celor două forme de trăsături a rezultat un vector cu 308 dimensiuni, pe care sunt antrenate clasificatoarele clasice.

1. Trăsăturile CNN învățate automat - 256 de dimensiuni

Arhitectura LineCNN, produce un vector de 256 de valori la penultimul strat înainte de clasificator, adică după AdaptiveAvgPool. Aceste trăsături sunt extrase prin mecanismul standard de “feature extraction”, și anume, imaginea mea este propagată prin rețeaua antrenată, iar activările penultimului strat sunt capturate ca reprezentare semantică învățată.

Deoarece CNN-ul a fost antrenat pe sarcina țintă (clasificare în 5 clase), aceste trăsături sunt discriminative pentru problemă, și anume, proiecția spațiului 308-dimensional este implicit optimizată pentru a separa cele 5 clase.

2. Trăsăturile manuale - 52 de dimensiuni

Aceste trăsături sunt extrase prin algoritmi clasici de procesare a imaginii, organizate în 6 familii, fiecare motivată de o ipoteză diferită despre structura datelor:

Tabelul 1: familiile de trăsături manuale

Familia	Trăsături	Număr	Motivație
A. Proiecții pe axe	mean, std, max, min pe coloane și pe rânduri	8	captează energia totală a semnalului în direcțiile principale
B. Peak-uri pe proiecții	număr, înălțimi, distanțe inter-peak, la 3 niveluri de proeminență	14	proxy pentru numărul de linii distinctive
C. Filtre Sobel direcționale [20]	magnitudini gradienti orizontali/verticali + raport	7	detectarea muchiilor (anume margini de linii) și orientarea lor dominantă
D. Histograma orientărilor	8 bin-uri pe intervalul $[0^\circ, 180^\circ]$ ponderate cu magnitudinea	8	distribuția orientărilor liniilor (HOG simplificat)
E. Statistici globale	medie, std, min, max + 6 percentile (25, 50, 75, 90, 95, 99)	10	caracteristici de distribuție a intensităților
F. Densități de pixeli aprinși	proporție pixeli > percentilele 70, 80, 85, 90, 95	5	“cât de aglomerată” este imaginea la diferite praguri

Pentru familia B (Peak-urile pe proiecții), am folosit *scipy.signal.find_peaks* cu trei niveluri diferite de proeminență:

- 0.05 - peak-uri slabe, captează liniile abia vizibile, cele foarte subțiri
- 0.10 - peak-uri medii, regimul “standard”, liniile normale
- 0.25 - peak-urile puternice, care captează liniile clare

Această stratificare o să permită modelului să distingă mult mai bine între tiparul liniilor, liniile “clare” și liniile “slabe” din aceeași imagine.

Pentru familia D (Histograma orientărilor cu HOG simplificat [19]), o să calculez histograma orientărilor gradientului pe 8 bin-uri uniforme pe $[0^\circ, 180^\circ]$, ponderată cu magnitudinea gradientului și filtrată să includă doar pixeli cu magnitudine peste percentila 75, astfel ignoră zgomotul slab.

Combinarea trăsăturilor

Vectorul final de 308 dimensiuni este obținut prin concatenarea celor 256 de trăsături de la CNN și cele 52 de trăsături manuale, fiecare grup fiind normalizat independent cu StandardScaler (media 0, deviația standard 1 pe fiecare dimensiune), fiind aplicat per-coloană pe setul de antrenare (pentru statistici fit-uite pe OOF, aplicate apoi pe test). Am normalizat separat deoarece cele două grupuri au scări inițiale diferite (trăsăturile CNN au valori în jurul lui 0 cu varianță mică, iar trăsăturile manuale au scări variate), iar dacă nu aș fi normalizat dimensiunile cu varianță mai mare ar fi dominat clasificatorul.

Am evaluat performanța celor două grupuri separat și combinat, prin antrenarea unui SVM RBF pe fiecare variantă cu CV 3-fold pe OOF și am obținut următoarele valori:

- doar pentru trăsăturile manuale: 0.5006 acuratețe
- doar pentru trăsăturile CNN: 0.7381 acuratețe
- trăsăturile CNN și trăsăturile manuale combinate: 0.7316 acuratețe

Rezultatul obținut m-a dus la o interpretare puțin contraintuitivă: trăsăturile manuale singure nu performează bine, iar aparent combinația cu CNN nu îmbunătățește SVM-ul singur. Cu toate acestea, în contextul ensemble-ului, trăsăturile manuale contribuie prin diversitatea predicțiilor, nu prin precizia individuală, de aceea le-am păstrat. Acest fenomen în care componente individuale mediocre îmbunătățesc agregatul este caracteristic ansamblurilor robuste.

Arhitectura CNN - Rețeaua convoluțională LineCNN

Am construit o arhitectură specifică pentru această sarcină, antrenată de la zero (nu am inclus greutatea preantrenate). Structura pe care am definit-o conține 4 blocuri convoluționale cu numere progresive de canale (de la 32, la 64, la 128, la 256), fiecare bloc având 2 straturi *Conv2d 3x3* [4] urmate de *BatchNorm2d* [6], *ReLU* și *MaxPool2d 2x2*. Pe ultimul bloc am aplicat *Dropout2d(0.2)* [7]. La final, *AdaptiveAvgPool2d(1)* [5] a redus reprezentarea spațială la un vector de 256 de valori, care o să intre într-un clasificator MLP cu două straturi liniare și dropout (0.5 și 0.3).

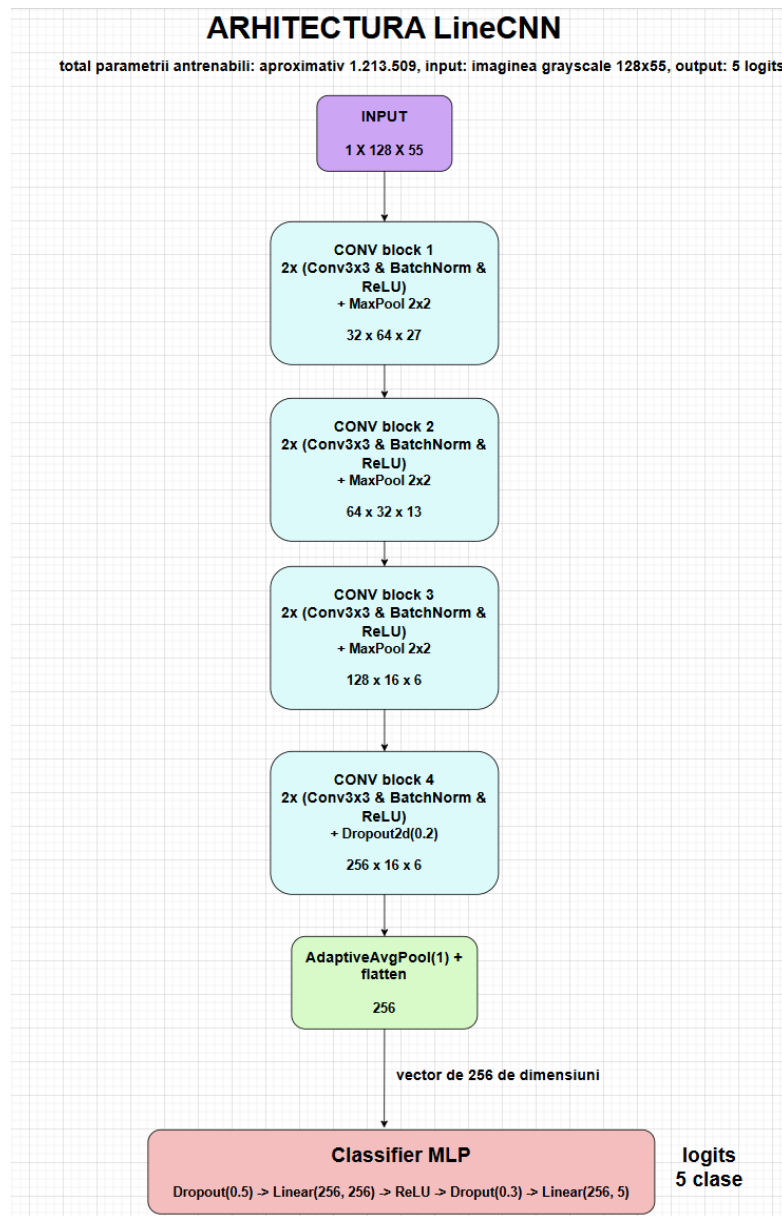


FIGURA 2: arhitectura LineCNN implementată cu dimensiunea tensorilor la fiecare etapă

Fiecare bloc convoluțional are formula:

$$\text{ConvBlock}(x) = \text{ReLU}(\text{BatchNorm}(\text{Conv}_{3 \times 3}(\text{ReLU}(\text{BatchNorm}(\text{Conv}_{3 \times 3}(x)))))$$

- două aplicații succesive de *Conv2d 3x3* cu padding egal cu 1, fiecare urmată de *BatchNorm2d* și *ReLU*. Această structură asigură următoarele aspecte:
- receptive field crescut progresiv - cele două straturi 3x3 consecutive au receptive field efectiv 5x5, păstrându-se astfel neliniaritatea suplimentară
- stabilitate la antrenare - *BatchNorm* normalizează activările și permite rate de învățare mai mari

Dimensiunile tensorului prin rețea pentru inputul 1 x 128 x 55:

1. input: 1 x 128 x 55
2. ConvBlock 1 (32 de canale) + MaxPool 2x2: 32 x 64 x 27
3. ConvBlock 2 (64 de canale) + MaxPool 2x2: 64 x 32 x 13
4. ConvBlock 3 (128 de canale) + MaxPool 2x2: 128 x 16 x 6
5. ConvBlock 4 (256 de canale) + Dropout2d(0.2): 256 x 16 x 6
6. AdaptiveAvgPool2d(1) + flatten: 256 valori

Pe ultimul bloc convoluțional nu o să aplic *MaxPool*, ci doar *Dropout2d(0.2)*, urmat de *AdaptiveAvgPool2d(1)*, care reduce reprezentarea spațială la un singur scalar pe canal. Am făcut această alegere de design deoarece are două avantaje:

1. *AdaptiveAvgPool* o să producă o reprezentare invariantă la mici translații spațiale, ceea ce devine util pentru sarcina de numărare, deoarece poziția exactă a unei linii nu contează de fapt, contează efectiv doar prezența ei
2. o să producă un vector de dimensiune fizică de 256, care este independent de micile variații ale dimensiunii inputului.

După startul *AdaptiveAvgPool*, vectorul de 256 de valori este parsat printr-un clasificator MLP cu 2 straturi liniare:

$$\text{Classifier}(z) = W_2 \times \text{ReLU}(W_1 \times z + b_1) + b_2$$

cu *Dropout(0.5)* aplicat înaintea primului strat și *Dropout(0.3)* între cele două straturi. Dimensiunile o să devină acum 256 -> 256 -> 5, outputul final fiind de 5 logits, transformați în probabilități prin softmax la inferență.

Numărul total de parametri antrenabili este de aproximativ 1.21 de milioane, mai puțin față de arhitecturile moderne. Am făcut această alegere deoarece pe imagini mici ca cele din problemă, modele mai complexe ar duce la overfitting.

Modele de clasificare și strategia de ensemble folosită

Pentru a îndeplini cerința proiectului privind utilizarea a minim două modele de învățare automată diferite, și pentru a explora atât abordarea modernă de deep learning, cât și cea clasică de învățare statistică, am implementat și testat 5 modele de clasificare, distribuite în două pipeline-uri:

1. CNN LineCNN - descris deja în secțiunea anterioară.

2. SVM cu kernel RBF [12], [13]

Clasificator antrenat pe trăsăturile combinate (cele cu 308 dimensiuni). Hiperparametrii au fost determinați prin grid search pe $C \in \{1, 10, 100\}$ și $\gamma \in \{scale, auto\}$, cu validare prin 3-fold cross-validation. Combinația optimă obținută a fost cea cu $C=1$, $\gamma=scale$.

3. MLP

Clasificator scikit-learn - *MLPClassifier* [11], antrenat pe aceleași trăsături combinate. Am folosit grid search pe 5 configurații de arhitectură, începând de la (256,) la (512, 256, 64) ori 2 valori $\alpha \in \{1e-4, 1e-3\}$ (am aplicat coeficient L2 pe greutatea). Am folosit optimizatorul implicit Adam, cu learning rate init 1e-3, maximul de iterații 300 și early stopping (se va opri când scorul de validare internă nu se mai îmbunătățește după 10 epoci consecutive).

4. KNN și Naive Bayes

Au fost implementate în pipeline-ul clasic alternativ (*solutie_ineficienta.py*). KNN folosește distanța euclidiană *weights = 'distance'*. Am făcut grid search pe $K \in \{3, 5, 7, 11, 15, 21, 31\}$. Am folosit *GaussianNB* din *scikit-learn*, fără alți parametri de reglat. L-am folosit ca baseline simplu în pipeline-ul clasic. [14], [15]

5. Funcția de pierdere - CrossEntropy cu label smoothing

CNN-ul a fost antrenat cu CrossEntropy Loss cu label smooting [8] (am ales coeficientul 0.05 pentru factor). Label smoothing previne nivelul de overconfidence al modelului și acționează ca o formă suplimentară de regularizare.

6. Strategia de ensemble - am folosit Soft Voting

Predicțiile celor trei modele principale, CNN, SVM și MLP le-am agregat prin soft voting [16] cu ponderile w_{CNN} , w_{SVM} , w_{MLP} , care au fost determinat prin căutarea exhaustivă pe grid (pas 0.05) pe predicțiile OFF din train. Am folosit constrângerea $w_{CNN} + w_{SVM} + w_{MLP} = 1$ ca să reduc spațiul la 2 grade de libertate. Pentru soluția principală pe care am obținut-o, am avut ponderile optime de:

$$w_{CNN} = 0.55, w_{SVM} = 0.30 \text{ și } w_{MLP} = 0.15.$$

Detalii despre antrenarea și strategia OOF

Hiperparametrii CNN:

Parametru	Valoare	Justificare
dimensiunea batch-ului	64	compromis între stabilitatea gradientului și viteză
numărul de epoci maxime	45	suficient pentru convergență, mai mult duce la overfit
patience early stopping	13	permite astfel rularea fără oprirea prematură
numărul minim de epoci	18	se garantează astfel că modelul nu se oprește înainte de a explora suficient
optimizatorul	Adam	oferă convergență rapidă, robust la learning rate
learning rate inițial	1e-3	standard pentru Adam
weight decay cu L2	1e-4	regularizare ușoară
netezirea etichetelor (label smoothing)	0.05	ca să previn overconfidence-ul
dropout (clasificator)	0.5, 0.3	o regularizare suplimentară

Programarea learning rate-ului folosind OneCycleLR

Am folosit scheduler-ul *OneCycleLR* [10] cu maximul learning rate-ului $3e-3$, ca să implementez strategia de antrenare „1-cycle”: în prima parte a antrenării, adică 30% din pași, learning rate-ul crește de la $max_lr/25$ la max_lr (faza de warmup), apoi descrește prin cosine annealing la o valoare foarte mică (faza de fine-tuning). Această programare permite explorarea mai în detaliu a spațiului parametrilor în prima parte, urmată de convergență fină în partea finală.

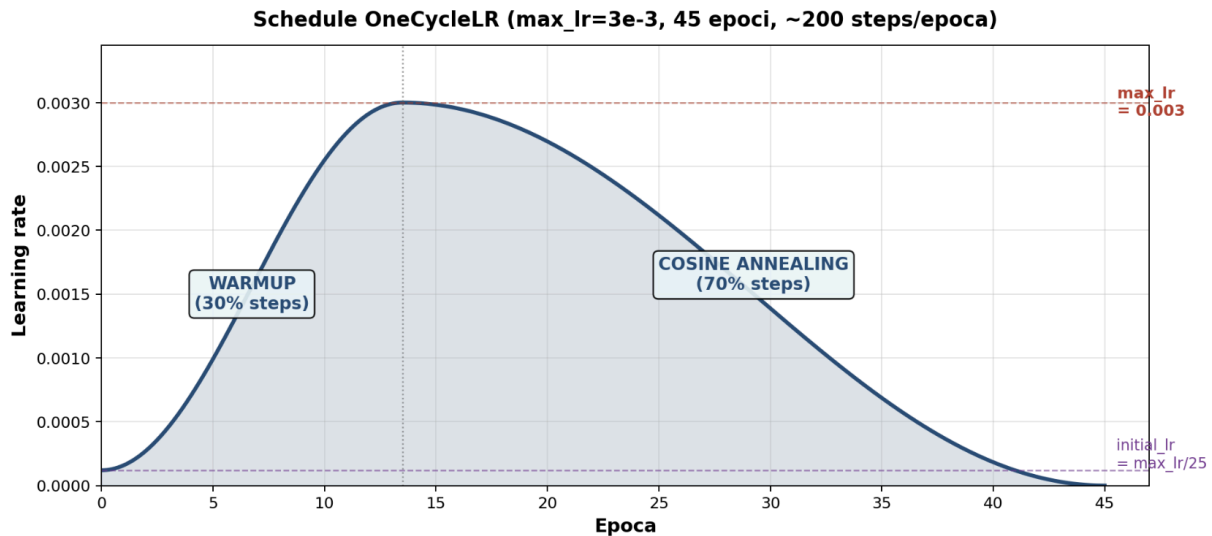


FIGURA 3: schedule OneCycleLR pe 45 de epoci

Strategia Out-of-Fold (OOF) folosită [18]

Am împărțit setul de antrenare în 5 folduri stratificate folosind *StratifiedKFold(n_splits=5, shuffle=True, random_state=42)* [23], apoi am repetat antrenarea cu 2 seed-uri diferite *seeds=[42, 123]*, pentru un total de 10 modele CNN antrenare. Strategia această folosită:

- previne data leakage între CNN și clasificatoarele de stack SVM și MLP - acestea nu ar vedea niciodată trăsăturile produse de un model care a memorat efectiv exemplele
- furnizează predicții oneste pentru întregul set de antrenare, permițând astfel o estimare mai realistă a performanței finale
- reduce varianța prin medierea peste aceste 2 seed-uri

Pentru predicția finală pe setul de test, am mediat predicțiile tuturor celor 10 modele (5 folduri pe 2 seeduri), aplicând și Test-Time Augmentation (TTA). Pentru fiecare imagine din setul de test, am mediat predicția pe imaginea originală cu predicția pe imaginea cu flip-ul orizontal.

$$p^{TTA} = (p(x) + p(\text{flip}(x)))/2$$

Variantele de rezolvări implementate și rezultatele obținute

Mai jos o să prezint evoluția abordărilor pe care le-am implementat pentru problemă, cât și performanța lor obținută. Am implementat și evaluat 10 variante distincte ale pipeline-ului. În tabel o să prezint explicit cele 8 versiuni cele mai semnificative (versiunile 5 și 6, bazate pe transformata Hough [21], [22], respectiv pipeline-ul pur clasic o să le discut mai jos):

Tabelul 2: comparația tuturor versiunilor testate

Versiune	Abordarea principală folosită	OFF	Kaggle public	Kaggle privat
versiunea 1	am folosit un CNN simplu, fără o preprocesare specializată	0.7229	0.73963	0.73793
versiunea 2 cu regresie	CNN cu MSE pentru pierdere, iar predicția era realizată prin rotunjire	0.5535	0.61236	0.58012
versiunea 3	CNN cu preprocesare adecvată liniilor	0.7449	0.78181	0.77066
versiunea 4	CNN mai adânc, cu convoluții orientate	0.7239	0.76581	0.75054
versiunea 7 (principală)	CNN cu trăsăturile manuale și SVM/MLP	0.7486	0.78545	0.77478
versiunea 8	am îmbunătățit versiunea 7 anterioară cu 4 seed-uri și adăugând modelul KNN	0.7493	0.78109	0.77284
versiunea 9	am adăugat la versiunea 7 Mixup și zgomot Gaussian	0.7348	nu am mai încărcat pe Kaggle	-
versiunea 10	am adăugat la versiunea 7 TTA extins cu 5 transformări	0.7480	0.78472	0.77042

Cele 3 soluții marcate colorat au fost cele care mi-au generat *submission_ensemble* pe care le-am selectat pentru leaderboard-ul final.

Îmbunătățirea soluțiilor a venit atunci când am trecut de la versiunea 1 la versiunea 3, când am realizat că introducerea preprocesării care scoate liniile din zgomotul de fundal, adică utilizarea contrast stretching pe percentile și normalizarea pe imagine, ajută mult mai mult la acuratețe. Această modificare a ridicat acuratețea OOF cu peste 2 procente, fără niciun alt fel de schimbare arhitecturală. Toate îmbunătățirile ulterioare aplicate, și anume, combinarea cu trăsături manuale, ensemble, TTA, mediere pe seed-uri, au adus câștiguri sub 1%, sugerând că preprocesarea a fost intervenția cu cel mai mare raport de beneficiu pentru problemă.

Tabelul 3: grid search-ul SVM (CV 3-fold pe trăsături CNN și cele manuale)

C	y	Acuratețe CV
1	scale	0.7437 (best)
1	auto	0.7436
10	scale	0.7316
10	auto	0.7317
100	scale	0.6866
100	auto	0.6866

Observație: C = 100 a produs overfit clar, deoarece acuratețea a scăzut prea grav la 0.6866, și mi-a confirmat că regularizarea contează pentru sarcina problemei. Valorile y = scale și y = auto, au produs aproximativ rezultate aproape identice, ce mi-a indicat că modelul este robust la alegerea exactă a lățimii kernelului.

Tabelul 4: grid search-ul MLP (CV 3-fold pe trăsături CNN și cele manuale)

Hidden layers	α	Acuratețe CV
(256,)	1e-4	0.7152
(256,)	1e-3	0.7210
(512, 128)	1e-4	0.7268 (best)
(512, 128)	1e-3	0.7244
(512, 256, 128)	1e-3	0.7218

Observație: arhitectura optimă obținută (512, 128) cu regularizare slabă $\alpha = 10^{-4}$ sugerează că două straturi ascunse sunt suficiente, iar regularizarea mai puternică nu a ajutat efectiv.

Tabelul 5: căutarea ponderilor ensemble pe OOF (varianta principală - adică versiunea 7, din grid search exhaustiv cu pas 0.05)

pondere CNN	pondere SVM	pondere MLP	acuratețe OOF
1.00	0.00	0.00	0.7470
0.70	0.30	0.00	0.7478
0.60	0.25	0.15	0.7481
0.55	0.30	0.15	0.7486 (best)
0.50	0.30	0.20	0.7484

Observație: ponderea dominantă pe CNN, fiind 0.55, reflectă faptul că este cel mai puternic clasificator individual, dar contribuțiile semnificative ale SVM de 0.30 și MLP de 0.15, demonstrează că ansamblul beneficiază real de diversitatea acestora.

Matricile de confuzie

Matricile de confuzie pe predicțiile OOF mi-au arătat structura consistentă a greșelilor în toate modelele. Mai jos este matricea de confuzie pentru ensemble-ul soluției principale (cu care am obținut o acuratețe de 0.7486):

	prezis clasa 1	prezis clasa 2	prezis clasa 3	prezis clasa 4	prezis clasa 5
real clasa 1	3344	119	10	8	19
real clasa 2	595	2222	100	42	41
real clasa 3	323	371	2063	171	72
real clasa 4	233	134	410	1931	292
real clasa 5	166	83	158	550	2043

Am descoperit două observații importante ce reies din această matrice de confuzie:

1. Erorile sunt aproape exclusiv între clase adiacente. Pentru clasa 4 reală, modelul prezice 4 corect în 64% din cazuri, iar greșelile sunt 14% spre clasa 3 și 10% spre clasa 5. Greșelile mai grave, cum ar fi clasa 4 prezisă drept clasa 1 sau clasa 2, sunt sub 8% cumulativ. Acest tipar este relativ natural pentru o problemă de numărare, deoarece este mai grea distingerea dintre 3 sau 4 obiecte, dar este mai ușoară distingerea dintre 5 sau 1 obiect.
2. Subnumărarea este sistematică spre clasa 1. Recall-ul clasei 1 este de 0.96, dar precision-ul este doar 0.72. Asta înseamnă că aproximativ 28% din imaginile prezise ca aparținând clasei 1 sunt de fapt din celelalte clase 2-5. Am observat că această tendință a modelului de a categorisi spre cea mai populată clasă a persistat în toate variantele pe care le-am implementat și testat, ceea ce demonstrează că diferențierea claselor cu număr mic de obiecte are o limită fundamentală în date.

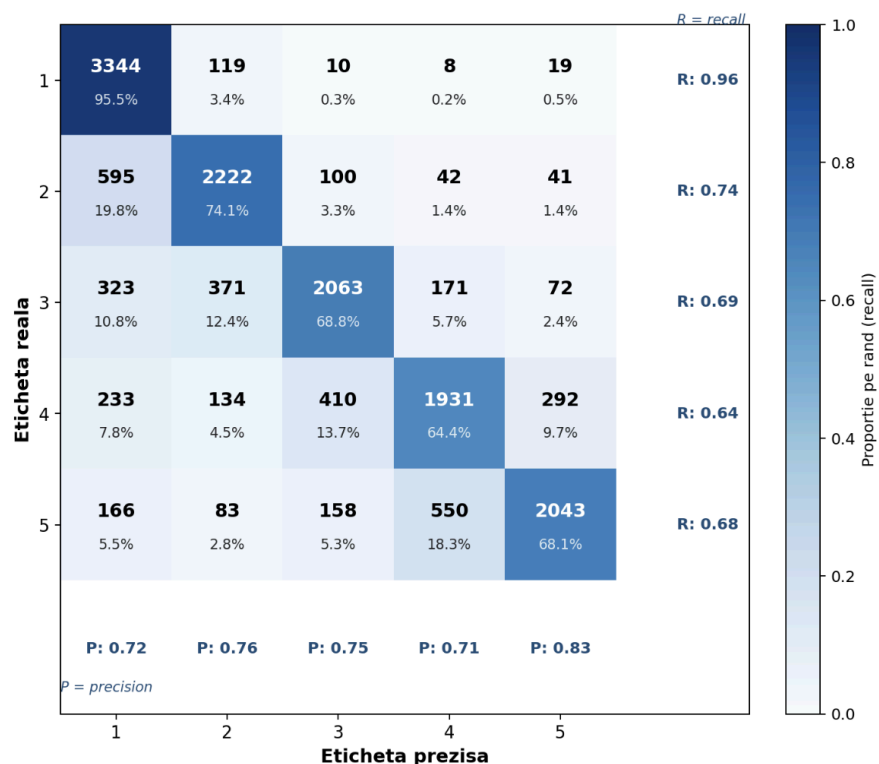
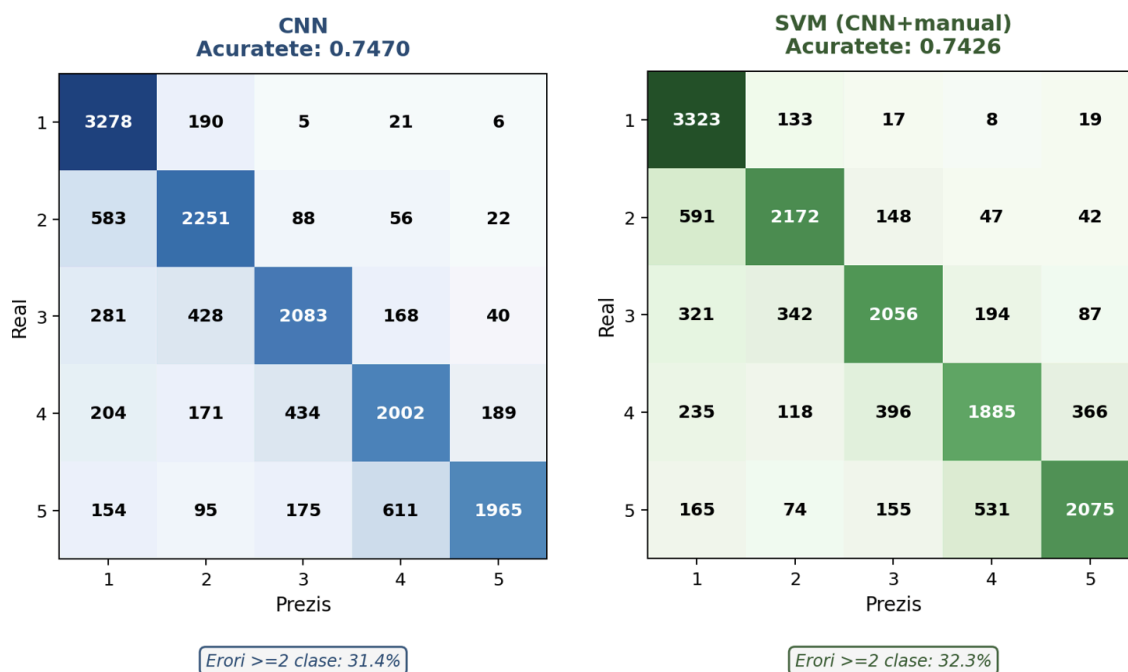
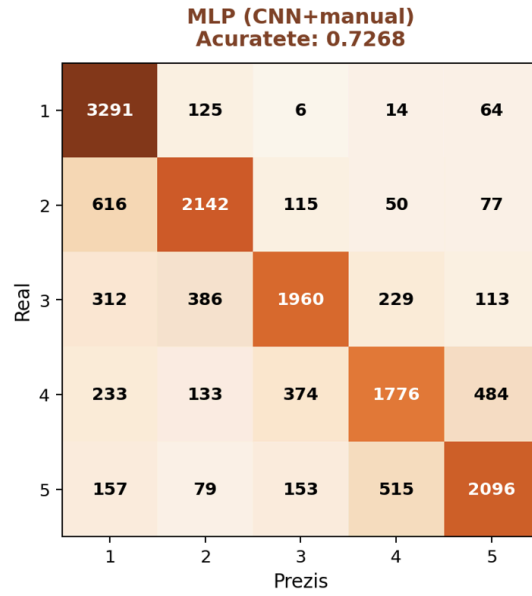


FIGURA 4: matricea de confuzie ensemble pentru versiunea 7 - principală OFF
 Diagonala dominantă confirmă predicția corectă în majoritatea cazurilor, iar valorile precision (P) și recall (R) afișate pe margini evidențiază tendința modelului de a categorisi predicțiile spre clasa 1, ce are recall-ul 0.96 și precision 0.72.





Erori ≥ 2 clase: 32.8%

FIGURA 5: matricile de confuzie pentru modelele CNN, SVM, MLP individuale ale ansamblului. Procentul de erori mai grave, adică la mai mult de 2 clase distanță, este afișat sub fiecare matrice și demonstrează că structura greșelilor (concentrată pe clase adiacente) este consistentă între modele.

Abordări ale problemei nereușite

Tratarea problemei ca regresie (versiunea 2 cu regresie)

Am antrenat un CNN cu funcția de pierdere MSE, în care eticheta este o valoare reală, de la 1.0 până la 5.0, iar predicția finală am obținut-o prin rotunjirea la cel mai apropiat întreg. M-am ghidat la această abordare deoarece greșelile între clase adiacente sunt mai puțin grave decât între clasele îndepărtate, iar regresia ar fi putut să exploateze structura ordinală a etichetelor. După ce am rulat și testat această abordare, am obținut un rezultat inferior clasificării, de aceea am renunțat la idee. Acest inconvenient a apărut deoarece MSE penalizează deviațiile, dar fără a maximiza explicit probabilitatea clasei corecte, de aceea, pentru metrici de tip accuracy, clasificarea cu CrossEntropy este mult mai potrivită.

CNN mai adânc cu convoluții orientate (implementat în versiunea 4)

Am încercat să adaug capacitate suplimentară modelului, și anume, un al 5-lea bloc convoluțional, de la 256 la 512 canale, kerneluri alungite, 7x1 și 1x7, în primul strat pentru a detecta explicit liniile verticale și orizontale, plus pooling asimetric (2x1 pe unele straturi) pentru a păstra rezoluția orizontală. Partea rea a venit când rezultatul s-a dovedit a fi mai prost decât arhitectura mai simplă inițială. Ca explicație probabilă pe imagini mici, capacitatea suplimentară a dus la overfitting, iar avantajul kernelurilor orientate nu a compensat suficient.

Trăsături bazate pe transformarea Hough (implementată în versiunea 5), dar abandonată.

Am încercat să folosesc și algoritmul Hough [21], [22], fiind algoritmul clasic pentru a detecta liniile drepte dintr-o imagine. Am considerat că numărul de segmente Hough să fie aproximativ numărul de obiecte pentru problema mea. Am testat pe 3 imagini, însă rezultatul a fost de asemenea prost (la 1 linie reală în imagine, algoritmul Hough a detectat 9-37 segmente, la 2 linii reale în imagine, a detectat 14-24 de segmente). Numărarea brută a eșuat din cauza zgomotului care fragmentează de fapt liniile reale în multe segmente mici. Am încercat și o variantă mai rafinată prin gruparea segmentelor după unghiul dominant, care a arătat o corelare mai bună, însă era tot zgomotoasă. Am abandonat această abordare înainte de a construi un cod complet, deoarece testele inițiale mi-au arătat că nu ar funcționa pentru problemă.

Mixup și Gaussian Noise (implementat în versiunea 9) - am obținut ca acuratețe 0.7348 OOF, cu aproximativ 1.4% sub versiunea 7, cea principală.

Mixup [9] am descoperit că este o tehnică modernă de regularizare în care două imagini din batch sunt combinate liniar $\lambda \cdot \text{img_A} + (1-\lambda) \cdot \text{img_B}$ cu etichete soft corespunzătoare. Pentru clasificare, Mixup ar îmbunătăți generalizarea, însă în cazul soluției implementate, a stricat performanța. Pentru problemă nu a funcționat deoarece sarcina principală era una de numărare, iar eticheta soft (0.5, clasa 2) + (0.5, clasa 4) nu corespunde de fapt realității fizice (2.5 obiecte nu reprezintă o imagine validă). Modelul implementat a învățat să prezică distribuții amestecate care nu generalizează la imagini reale. Mi s-a părut chiar interesantă această observație, realizând că nu orice tehnică de regularizare bună este transferabilă între diferite tipuri de sarcini.

Discuție finală

Cele 10 variante pe care le-am implementat, cât și testat, dintre care 4-5 au atins acuratețea OOF în intervalul îngust 0.745-0.749, mi-au sugerat faptul că nu am reușit să mă ridic peste plafonul empiric al datelor cu modelele și modul în care le-am implementat. Cele trei abordări pe care le-am implementat, fiind fundamental diferite, și anume, CNN curat, CNN și caracteristici geometrice Hough, CNN și caracteristici manuale și ensemble clasic, au atins aproximativ același plafon. Diferența între variante a fost doar din pricina zgomotului statistic, versiunile cu OFF 0.7493 (versiunea 8) sau 0.7486 (versiunea 7) sau 0.7480 (versiunea 10) nu am putut să le diferențiez semnificativ pe baza unui singur experiment.

Tiparul greșelilor apărute, cum ar fi confuziile între clasele adiacente și subnumărarea spre clasa 1, a apărut în toate modelele implementate și testate, indiferent de arhitectură sau de tehnică. De aceea, consider că limita observată este fundamentală în date, nu în metode (probabil există exemple ambigue în setul de test, în care liniile se suprapun sau sunt prea slabe pentru a fi numărate sigur).

Soluția mea finală combină ceea ce a funcționat cu adevărat în cazul meu:

- o preprocesare adecvată pentru tipul de semnal (liniile subțiri pe fundalul zgomotos) - contribuția cu cel mai mare impact, aproximativ 2% acuratețe OOF
- un CNN antrenat de la zero pentru extragerea automată de trăsături pentru a evita overfitting-ul pe imagini mici
- mediere peste 10 modele, 2 seed-uri cu 5 folduri stratificate, cu strategie out-of-fold riguroasă pentru a preveni data leakage
- un set complementar de 52 de trăsături manuale bazate pe structura geometrică a semnalului (proiecții, peak-uri, Sobel, HOG simplificat)
- ansamblu prin soft voting cu ponderi optimizate pe predicții OOF, valorificând astfel diversitatea celor 3 modele, CNN, SVM și MLP
- Test-Time Augmentation prin medierea între imaginea originală și varianta flipuită orizontal

Acuratețea finală obținută este de 0.78545 pe leaderboard-ul public Kaggle, și 0.77478 pe leaderboard-ul privat.

Anexă

Codul soluției principale (versiunea 7 a codului) este organizat în 5 fișiere .py la rădăcina proiectului Dragunoi_Miruna_244:

- [configurare.py](#) - toate constantele și căile
- [preprocesare.py](#) - am implementat funcția de preprocesare() care implementează cei 3 pași de preprocesare discutați
- [features.py](#) - aici a fost implementată efectiv extragerea celor 52 de trăsături manuale, cu cache .npy pentru reutilizare
- [cnn.py](#) - am utilizat clasa SignalDataset din PyTorch, arhitectura LineCNN și funcțiile de infereaza() și train_un_fold(). Am adăugat și funcția infereaza_tta_extinsa() pentru implementarea versiunii 10 de cod în care am folosit 5 transformări (imaginea originală, imaginea cu flip spre orizontală, imaginea cu translație spre stânga cu -2 pixeli pe axa orizontală, imaginea cu translația spre dreapta cu +2 pixeli pe axa orizontală și imaginea cu flipul orizontal combinat cu translația spre stânga).
- [main.py](#) - programul principal pentru soluția principală (versiunea 7), în care se rulează tot pipeline-ul

Am inclus suplimentar și următoarele fișiere (pentru a demonstra modul de lucru pentru versiunile 8 și 10 care au fost selectate pentru leaderboard-ul final, cât și versiunea clasică de cod, dar ineficientă, de la care am plecat):

- [solutie_ineficienta.py](#) - pipeline-ul alternativ discutat anterior, pur clasic, care implementează efectiv Naive Bayes, KNN, SVM, MLP pe trăsăturile manuale, dar fără CNN, pe care l-am folosit ca linie de bază
- [main_versiunea8.py](#) - pentru a exemplifica îmbunătățirea versiunii 7 de cod, suplimentar adăugat fiind doar 4 seed-uri și KNN adăugat în ensemble
- [main_versiunea10.py](#) - pentru a exemplifica îmbunătățirea versiunii 7 de cod prin adăugarea TTA extinsă

Bibliografie:

- [1] Cursurile și Laboratoarele de Inteligență Artificială (Învățare Automată) - Artificial Intelligence Lectures, Spring 2024 (Bachelor), Faculty of Mathematics and Computer Science, University of Bucharest - <https://fmi-unibuc-ia.github.io/ia/>
- [2] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, „Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, nr. 11, pp. 2278–2324, 1998.
- [3] A. Krizhevsky, I. Sutskever, G. E. Hinton, „ImageNet Classification with Deep Convolutional Neural Networks,” *Advances in Neural Information Processing Systems (NIPS)*, 2012.
- [4] K. Simonyan, A. Zisserman, „Very Deep Convolutional Networks for Large-Scale Image Recognition,” *International Conference on Learning Representations (ICLR)*, 2015. - pentru arhitectura cu blocuri de două convoluții 3×3 succesive
- [5] M. Lin, Q. Chen, S. Yan, „Network In Network,” *International Conference on Learning Representations (ICLR)*, 2014. - pentru Global Average Pooling (AdaptiveAvgPool2d)
- [6] S. Ioffe, C. Szegedy, „Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” *International Conference on Machine Learning (ICML)*, 2015.
- [7] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, „Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.
- [8] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, Z. Wojna, „Rethinking the Inception Architecture for Computer Vision,” *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. - pentru Label Smoothing
- [9] H. Zhang, M. Cisse, Y. N. Dauphin, D. Lopez-Paz, „mixup: Beyond Empirical Risk Minimization,” *International Conference on Learning Representations (ICLR)*, 2018.
- [10] L. N. Smith, N. Topin, „Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates,” *arXiv preprint arXiv:1708.07120*, 2017. - pentru OneCycleLR
- [11] D. E. Rumelhart, G. E. Hinton, R. J. Williams, „Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [12] C. Cortes, V. Vapnik, „Support-Vector Networks,” *Machine Learning*, vol. 20, nr. 3, pp. 273–297, 1995. - SVM
- [13] B. E. Boser, I. M. Guyon, V. N. Vapnik, „A training algorithm for optimal margin classifiers,” *Proceedings of the Fifth Annual Workshop on Computational Learning Theory*, pp. 144–152, 1992. - kernel trick / RBF

- [14] T. Cover, P. Hart, „Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, nr. 1, pp. 21–27, 1967. - KNN
- [15] R. O. Duda, P. E. Hart, D. G. Stork, *Pattern Classification*, ediția a 2-a, Wiley, 2001. - referință clasică pentru Naive Bayes Gaussian și fundamente ML
- [16] T. G. Dietterich, „Ensemble Methods in Machine Learning," *International Workshop on Multiple Classifier Systems*, pp. 1–15, 2000.
- [17] R. Polikar, „Ensemble based systems in decision making," *IEEE Circuits and Systems Magazine*, vol. 6, nr. 3, pp. 21–45, 2006.
- [18] D. H. Wolpert, „Stacked generalization," *Neural Networks*, vol. 5, nr. 2, pp. 241–259, 1992. - fundamentele strategiei de stacking cu predicții OOF
- [19] N. Dalal, B. Triggs, „Histograms of Oriented Gradients for Human Detection," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005. - HOG
- [20] I. Sobel, G. Feldman, „A 3×3 Isotropic Gradient Operator for Image Processing," prezentat la Stanford Artificial Intelligence Project (SAIL), 1968. - filtre Sobel
- [21] P. V. C. Hough, „Method and Means for Recognizing Complex Patterns," U.S. Patent 3.069.654, 18 dec. 1962.
- [22] R. O. Duda, P. E. Hart, „Use of the Hough Transformation to Detect Lines and Curves in Pictures," *Communications of the ACM*, vol. 15, nr. 1, pp. 11–15, 1972. - Hough generalizat pentru linii
- [23] R. Kohavi, „A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection," *International Joint Conference on Artificial Intelligence (IJCAI)*, 1995. - Stratified K-Fold